

Lab 3 – Backtracking (Quay lui) với Python

1. Mục tiêu

Kết thúc bài thực hành, sinh viên có khả năng:

- Hiểu và áp dụng template backtracking 3 bước: Choose → Explore → Unchoose
- Viết được các hàm backtracking cơ bản (hoán vị, tổ hợp, tập con)
- Cài đặt được bài toán N-Queens với kiểm tra ràng buộc
- Áp dụng kỹ thuật pruning để tối ưu hiệu suất
- Đo lường và so sánh hiệu quả của pruning

2. Nội dung

Bài 1 – Backtracking cơ bản

Yêu cầu chung

Tạo file lab3_bai1.py, viết 4 hàm backtracking sau đây. Mỗi hàm phải có:

- Comment ghi rõ base case và recursive case
- In kết quả rõ ràng
- Test với ít nhất 2 trường hợp

Hàm 1 – Tìm tất cả hoán vị (Permutations)

Yêu cầu: Viết hàm `permutations(nums)` tìm tất cả cách sắp xếp của danh sách `nums`

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input: [1, 2, 3]

- Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
- Ý tưởng: Ở mỗi vị trí, chọn một số từ các số còn lại

Bước 2: Template backtracking

```
def backtrack(path, remaining):
    # Base case: Đã chọn đủ số
    if điều_kiện_dừng:
        lưu_kết_quả
        return

    # Thử từng lựa chọn
    for choice in choices:
        # CHOOSE
        path.append(choice)

        # EXPLORE
        backtrack(path_mới, remaining_mới)

        # UNCHOOSE
        path.pop()
```

Bước 3: Code hoàn chỉnh

```
def permutations(nums):
    """
    Tìm tất cả hoán vị của nums
    """
    result = []

    def backtrack(path, remaining):
        # Base case: Đã chọn đủ n số
```

```

    if len(path) == len(nums):
        result.append(path.copy()) # QUAN TRỌNG: phải copy!
        return

    # Thử từng số còn lại
    for i in range(len(remaining)):
        # CHOOSE: Chọn remaining[i]
        path.append(remaining[i])

        # EXPLORE: Đệ quy với các số còn lại (bỏ số vừa chọn)
        new_remaining = remaining[:i] + remaining[i+1:]
        backtrack(path, new_remaining)

        # UNCHOOSE: Quay lui
        path.pop()

    backtrack([], nums)
    return result

# Test
print("=== Test Permutations ===")
result1 = permutations([1, 2, 3])
print(f"Hoán vị của [1,2,3]: {result1}")
print(f"Số hoán vị: {len(result1)}") # Kỳ vọng: 6 (= 3!)

result2 = permutations([1, 2])
print(f"\nHoán vị của [1,2]: {result2}")
print(f"Số hoán vị: {len(result2)}") # Kỳ vọng: 2 (= 2!)

```

Bước 4: Giải thích

- `path`: Lưu hoán vị đang xây dựng
- `remaining`: Các số chưa dùng
- `path.copy()`: Tạo bản sao (nếu không copy sẽ lưu reference, tất cả đều giống nhau!)
- `remaining[:i] + remaining[i+1:]`: Tạo list mới không chứa phần tử `i`

Độ phức tạp:

- Thời gian: $O(N! \times N)$ - $N!$ hoán vị, mỗi hoán vị tốn $O(N)$ để copy
- Không gian: $O(N)$ cho call stack

Hàm 2 – Tìm tất cả tổ hợp (Combinations)

Yêu cầu: Viết hàm `combinations(nums, k)` tìm tất cả tổ hợp `k` phần tử từ `nums`

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input: `nums=[1,2,3,4]`, `k=2`
- Output: `[[1,2], [1,3], [1,4], [2,3], [2,4], [3,4]]`
- Khác với hoán vị: `[1,2]` và `[2,1]` là giống nhau (chỉ lấy 1 cái)

Bước 2: Chiến lược

- Để tránh trùng lặp, chỉ chọn các số **sau** số vừa chọn
- Dùng tham số `start` để đánh dấu vị trí bắt đầu

Bước 3: Code hoàn chỉnh

```
def combinations(nums, k):
    """
    Tìm tất cả tổ hợp k phần tử từ nums
    """
    result = []
```

```

def backtrack(start, path):
    # Base case: Đã chọn đủ k phần tử
    if len(path) == k:
        result.append(path.copy())
        return

    # Thử các số từ vị trí start trở đi
    for i in range(start, len(nums)):
        # CHOOSE
        path.append(nums[i])

        # EXPLORE: Chỉ xét các số sau i (i+1)
        backtrack(i + 1, path)

        # UNCHOOSE
        path.pop()

backtrack(0, [])
return result

```

Test

```

print("\n=== Test Combinations ===")
result1 = combinations([1, 2, 3, 4], 2)
print(f"Tổ hợp 2 từ [1,2,3,4]: {result1}")
print(f"Số tổ hợp: {len(result1)}") # Kỳ vọng: 6 (C(4,2) = 6)

```

```

result2 = combinations([1, 2, 3], 2)
print(f"\nTổ hợp 2 từ [1,2,3]: {result2}")
print(f"Số tổ hợp: {len(result2)}") # Kỳ vọng: 3 (C(3,2) = 3)

```

Bước 4: Giải thích

- start: Vị trí bắt đầu trong nums
- i + 1: Chỉ xét các số sau vị trí i → tránh trùng lặp
- Công thức tổ hợp: $C(n, k) = n! / (k! \times (n-k)!)$

Độ phức tạp:

- Thời gian: $O(C(n, k) \times k) = O(n! / ((n-k)! \times k!) \times k)$
- Không gian: $O(k)$ cho call stack

Hàm 3 – Tìm tất cả tập con (Subsets)

Yêu cầu: Viết hàm subsets(nums) tìm tất cả tập con của nums

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input: [1, 2, 3]
- Output: [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]
- Tổng số tập con: 2^n (mỗi phần tử chọn hoặc không chọn)

Bước 2: Chiến lược

- Giống combinations nhưng **không có điều kiện k**
- Lưu tất cả path gặp phải (không chỉ khi đủ k phần tử)

Bước 3: Code hoàn chỉnh

```
def subsets(nums):  
    """"  
    Tìm tất cả tập con của nums  
    """"  
    result = []
```

```

def backtrack(start, path):
    # Lưu tất cả tập con (không có điều kiện dừng cụ thể)
    result.append(path.copy())

    # Thử thêm các phần tử từ start
    for i in range(start, len(nums)):
        # CHOOSE
        path.append(nums[i])

        # EXPLORE
        backtrack(i + 1, path)

        # UNCHOOSE
        path.pop()

    backtrack(0, [])
    return result

```

```

# Test
print("\n=== Test Subsets ===")
result1 = subsets([1, 2, 3])
print(f"Tập con của [1,2,3]: {result1}")
print(f"Số tập con: {len(result1)}") # Kỳ vọng: 8 (= 2^3)

result2 = subsets([1, 2])
print(f"\nTập con của [1,2]: {result2}")
print(f"Số tập con: {len(result2)}") # Kỳ vọng: 4 (= 2^2)

```

Bước 4: Giải thích

- Không có base case rõ ràng (lưu tất cả path)
- Mỗi lần gọi backtrack đều lưu path hiện tại
- Tổng số tập con: 2^n

Độ phức tạp:

- Thời gian: $O(2^n \times n)$ - 2^n tập con, mỗi tập tốn $O(n)$ copy
- Không gian: $O(n)$ cho call stack

Hàm 4 – In tất cả chuỗi nhị phân độ dài n

Yêu cầu: Viết hàm `binary_strings(n)` in tất cả chuỗi nhị phân (0 và 1) có độ dài n

Hướng dẫn chi tiết:

Bước 1: Phân tích

- Input: `n = 3`
- Output: `['000', '001', '010', '011', '100', '101', '110', '111']`
- Tổng số: 2^n

Bước 2: Code hoàn chỉnh

```
def binary_strings(n):
    """
    Tìm tất cả chuỗi nhị phân độ dài n
    """
    result = []

    def backtrack(path):
        # Base case: Đủ n ký tự
        if len(path) == n:
            result.append(''.join(path))
```



```

        return

    # Thử cả '0' và '1'
    for choice in ['0', '1']:
        # CHOOSE
        path.append(choice)

        # EXPLORE
        backtrack(path)

        # UNCHOOSE
        path.pop()

    backtrack([])
    return result

# Test
print("\n=== Test Binary Strings ===")
result1 = binary_strings(3)
print(f"Chuỗi nhị phân độ dài 3: {result1}")
print(f"Số chuỗi: {len(result1)}") # Kỳ vọng: 8 (= 2^3)

result2 = binary_strings(2)
print(f"\nChuỗi nhị phân độ dài 2: {result2}")
print(f"Số chuỗi: {len(result2)}") # Kỳ vọng: 4 (= 2^2)

```

Bước 4: Giải thích

- Choices cố định: ['0', '1']
- ".join(path)": Ghép list thành string

- Tương tự cây quyết định: mỗi nút có 2 nhánh (0 hoặc 1)

Độ phức tạp:

- Thời gian: $O(2^n \times n)$
- Không gian: $O(n)$

Bài 2 – N-Queens với pruning

Trong bài này, sinh viên cài đặt bài toán N-Queens và tối ưu bằng pruning.

File: lab3_bai2.py

Phần A – Hàm kiểm tra hợp lệ

Bước 1: Copy code sau vào file

```
def is_safe(board, row, col, n):
    """
    Kiểm tra đặt quân hậu ở (row, col) có hợp lệ không
    board: list lưu cột của quân hậu ở mỗi hàng
    board[i] = j nghĩa là quân hậu hàng i ở cột j
    """
    # Kiểm tra tất cả các hàng trước đó
    for prev_row in range(row):
        prev_col = board[prev_row]

        # Kiểm tra cùng cột
        if prev_col == col:
            return False

    # Kiểm tra đường chéo
    # Nếu |row1 - row2| == |col1 - col2| → cùng đường chéo
    if abs(prev_row - row) == abs(prev_col - col):
```

```

        return False

    return True

def print_board(solution, n):
    """
    In bàn cờ N×N với quân hậu
    """
    for row in range(n):
        line = ""
        for col in range(n):
            if solution[row] == col:
                line += "Q "
            else:
                line += ". "
        print(line)
    print()

```

Phần B – N-Queens không có pruning

Yêu cầu: Cài đặt N-Queens **không kiểm tra** hợp lệ trước khi đệ quy

Code khung:

```

class Counter:
    """Class để đếm số lần gọi hàm"""
    def __init__(self):
        self.total_calls = 0
        self.solutions = 0

    def report(self):
        print(f"Tổng số lần gọi: {self.total_calls}")

```

```
print(f"Số giải pháp tìm được: {self.solutions}")
```

```
def solve_n_queens_no_pruning(n):  
    """  
    N-Queens KHÔNG có pruning (kiểm tra sau)  
    """  
    counter = Counter()  
    result = []  
    board = []  
  
    def backtrack(row):  
        counter.total_calls += 1  
  
        # Base case: Đã đặt đủ n quân hậu  
        if row == n:  
            # TODO: Kiểm tra board có hợp lệ không  
            # Nếu hợp lệ → lưu vào result  
            # counter.solutions += 1  
            return  
  
        # TODO: Thử tất cả cột (0 đến n-1)  
        # KHÔNG kiểm tra is_safe trước  
        for col in range(n):  
            # CHOOSE  
            board.append(col)  
  
            # EXPLORE  
            backtrack(row + 1)  
  
            # UNCHOOSE
```

```
board.pop()

backtrack(0)
counter.report()
return result
```

Yêu cầu sinh viên:

1. Hoàn thành TODO trong hàm backtrack
2. Kiểm tra board hợp lệ bằng cách duyệt tất cả các cặp quân hậu
3. Test với $n=4$ và $n=5$

Phần C – N-Queens có pruning

Yêu cầu: Cài đặt N-Queens **có kiểm tra** hợp lệ trước khi đệ quy

Code khung:

```
def solve_n_queens_with_pruning(n):
    """
    N-Queens CÓ pruning (kiểm tra trước)
    """
    counter = Counter()
    result = []
    board = []

    def backtrack(row):
        counter.total_calls += 1

        # Base case
        if row == n:
            result.append(board.copy())
            counter.solutions += 1
```

```

    return

# TODO: Thử từng cột
for col in range(n):
    # PRUNING: Kiểm tra is_safe TRƯỚC khi đệ quy
    if is_safe(board, row, col, n):
        # CHOOSE
        board.append(col)

        # EXPLORE
        backtrack(row + 1)

        # UNCHOOSE
        board.pop()

backtrack(0)
counter.report()
return result

```

Yêu cầu sinh viên:

1. Hoàn thành hàm `backtrack`
2. Đảm bảo gọi `is_safe()` trước khi đệ quy

Phần D – So sánh hiệu suất

Yêu cầu: Viết hàm `compare_n_queens(n)` để so sánh 2 phiên bản

Gợi ý:

```

import time

def compare_n_queens(n):
    print(f"\n{'='*50}")

```

```

print(f"So sánh N-Queens với N={n}")
print(f"{'='*50}")

# TODO: Test không có pruning
print("\n[1] KHÔNG có pruning:")
start = time.time()
result1 = solve_n_queens_no_pruning(n)
time1 = time.time() - start
print(f"Thời gian: {time1:.6f}s")

# TODO: Test có pruning
print("\n[2] CÓ pruning:")
start = time.time()
result2 = solve_n_queens_with_pruning(n)
time2 = time.time() - start
print(f"Thời gian: {time2:.6f}s")

# TODO: In kết quả so sánh
print(f"\nTốc độ tăng: {time1/time2:.2f}x")

# In một giải pháp mẫu
if len(result2) > 0:
    print(f"\nMột giải pháp cho {n}-Queens:")
    print_board(result2[0], n)

# Test
compare_n_queens(4)
compare_n_queens(6)

```

Bài 3 – Subset Sum với pruning nâng cao

Trong bài này, sinh viên cài đặt Subset Sum với nhiều kỹ thuật pruning.

File: lab3_bai3.py

Phần A – Subset Sum cơ bản

Yêu cầu: Tìm tất cả tập con có tổng bằng target

Code khung:

```
def subset_sum_basic(nums, target):  
    """  
    Subset Sum cơ bản - KHÔNG có pruning  
    """  
    result = []  
  
    def backtrack(start, path, current_sum):  
        # TODO: Base case - nếu current_sum == target  
  
        # TODO: Thử thêm các số từ vị trí start  
        for i in range(start, len(nums)):  
            # CHOOSE  
            path.append(nums[i])  
  
            # EXPLORE  
            backtrack(i + 1, path, current_sum + nums[i])  
  
            # UNCHOOSE  
            path.pop()  
  
    backtrack(0, [], 0)  
    return result
```

Sinh viên tự hoàn thành.

Phần B – Subset Sum với pruning

Yêu cầu: Thêm các kỹ thuật pruning sau:

1. Early termination:

```
if current_sum > target:  
    return # Dừng ngay, không đệ quy tiếp
```

2. Sort và break:

```
nums.sort() # Sắp xếp tăng dần  
...  
if current_sum + nums[i] > target:  
    break # Các số sau còn lớn hơn → break luôn
```

3. Skip duplicates:

```
if i > start and nums[i] == nums[i-1]:  
    continue # Bỏ qua số trùng
```

4. Remaining check:

```
remaining_sum = sum(nums[i:])  
if current_sum + remaining_sum < target:  
    break # Cộng hết cũng không đủ
```

Sinh viên tự cài đặt tất cả 4 kỹ thuật.

Phần C – So sánh và đo lường

Yêu cầu:

1. Cài đặt class `Counter` để đếm số nhánh
2. Tạo hàm `compare_subset_sum(nums, target)`
3. Test với input lớn: `nums = list(range(1, 20))`, `target = 50`
4. In báo cáo:
 - Tổng số lần gọi hàm

- Số nhánh đã cắt tỉa
- Tỷ lệ cắt (%)
- Thời gian chạy
- Tốc độ tăng (lần)

Sinh viên tự nghiên cứu và cài đặt.